Introduction to Architecture Exam

ExamTag: 2022-Jun

Instructions

Format: This exam consists of 6 multiple choice questions for each of the 12 lectures in the course. You may have to do some calculations in order to determine the correct answer. The motivation for having so many questions is to get a more accurate assessment of your knowledge of the material.

Be sure to circle the answers on the page titled "answers page" or your answers will not be graded. If you choose not to answer a question do not circle a letter. Feel free to detach the answers page from the remainder of the exam and only turn it in.

Grading: To discourage guessing, each incorrect answer will be worth -1/3 point, while each correct answer is worth +1 point. If you don't answer you will get 0 points for that question. I.e., if you guess randomly, the expected score is 0.

Material: You are allowed a calculator and one double-sided, hand-written A4 sheet of notes during the exam.

Good luck!

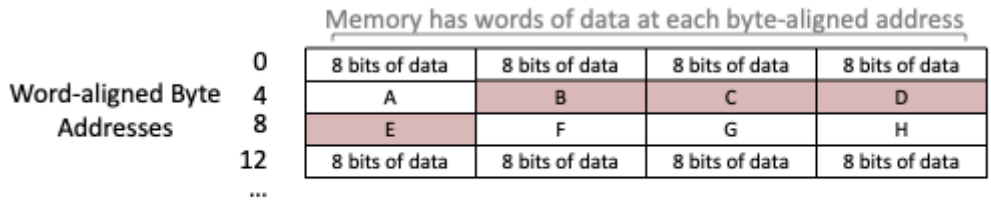
Note: Please take this exam seriously and pass it! (Exams take huge amounts of teaching time to produce and grade, and most students and teachers would prefer that the time be spent teaching instead.)

1. ISA 1

1. How many registers can you use to store data in a MIPS program?
    a. 32
    b. 31
    c. 24
    d. Depends on which registers the function calls use

2. What does the following code put in R3?
```
lw R1, 12(R0)
lw R2, 16(R0)
sll R1, R1, 16
slr R2, R2, 16
or R3, R1, R2
```
    a. Zero
    b. The ORed sum of the data at addresses 12 and 16
    c. The ORed sum of the data at addresses 12 and 16 multiplied by 216
    d. The word starting at address 14

3. MIPS does not support unaligned accesses. Below is a list of instructions in order with a brief explaination. Identify the *wrong* instruction or explaination when trying to load bytes B,C,D,E into register R3.



(The unaligned **word at address 5** contains bytes **B C D E**.)

    a. `lw R2, 4(R0)`: load word at address 4
    b. `sll R2, R2, 8`: (Shift by 1 byte)
    c. `lw R4, 8(R0)`: load word at address 8
    d. `or R3, R4, R2`: Combine R4 and R2 using bitwise-OR

4. Which register is most likely to be modified?
    a. R0
    b. A0
    c. RA
    d. PC

5. Why is it less efficient to compute directly on data in memory than data in registers?
    a. Main memory is slower than registers
    b. Main memory requires more address bits to access than registers
    c. A and B
    d. Computing on data in memory is as efficient as data in registers

6. In MIPS, what special meaning does the "exit:" label has compared to other labels?
    a. It must be used in branch instructions to jump to the end of a loop
    b. It must be used in branch instructions to jump to the end of the program
    c. a and b
    d. It has no special meaning

2. ISA 2

1. This is the last code in a procedure. What instructions should be placed in _____?
   lw $t0, 0($sp)
   lw $t1, 4($sp)
   _____
   _____
   a. addi $sp, $sp, 8; jr $ra
   b. addi $sp, $sp, -8; jal proceedure
   c. addi $sp, $sp, 2; jr $ra
   d. addi $sp, $sp, -2; jal proceedure

2. How does the processor know the instruction type?
   a. If the instruction has 0, 2, or 3 register fields
   b. If there is an immediate in the instruction
   c. The binary code contains instruction type bits for each instruction
   d. None of the above

3. What happens with the immediate for branch instructions, and why?
   a. It is shifted right by 8 bits because the lower byte is not necessary
   b. It is shifted right because instruction addresses are always multiples of 4
   c. It is shifted left because instructions are word-aligned
   d. It is not modified

4. Select the correct branch instruction that will complete the for loop: `for (i = 0; i != 10; i++)`.

```
add  $t0, $zero, $zero     # i = 0
addi $t1, $zero, 10        # j = 10
addi $t0, $t0, 1           # i++
slt  $t2, $t0, $t1         # t2=1 if (t0<t1) else 0
bne  $t2, $zero, _____     #  jump to ?
```

   a. bne $t2, $zero, -8
   b. bne $t2, $zero, -2
   c. bne $t2, $zero, -12
   d. bne $t2, $zero, -3

5. A dual-issue pipeline has one ALU that can take inputs from two registers or one register and the sign-extender, and sends its result back to the register file. The second ALU can take inputs from one register and the sign-extender, but only sends its result to the address input of the memory. A designer realizes that this is not ideal and changes it so either ALU can send their results back to the register file. Which pair of instructions can now be executed at the same time that could not be before? (Note, the list below does not imply any ordering. E.g., A and B does not mean that A has to execute on ALU 1 and B on ALU 2.)
   a. add and lw
   b. addi and lw
   c. add and addi
   d. lw and lw

6. A function uses registers $s0, $s1 and $t3. How much will it adjust the stack when it is called?
   a. -1
   b. -2
   c. -4
   d. -8

3. Computer Arithmetic

1. Base-2 arithmetic is what is used in our MIPS processor: numbers are stored in binary (1s and 0s) and each binary digit represents twice the value before it. Another approach is to use Base-10 number representations, where we use 4 binary digits together to represent the numbers 0-9. (Representing 0-99 would require two sets of 4 bits, one for the one's digit and one for the ten's digit, and representing 0.0 to 9.9 would require the same number of bits.) We also need to build Base-10 adders that can do the right thing with this representation. Why would one want to use base-10 arithmetic in computers instead of base-2 arithmetic? (Hint: IBM actually makes processors that have hardware for Base-10. Why would they do that?)
    a. It is easier for humans to reason about because we think in Base-10
    b. It is easier for computers to work with because carries are simpler in adders
    c. It is easier to avoid rounding errors with values like money that are already in Base-10
    d. None of the above

2. What would the following 2's complement math compute: (A+1) + (!B + !C)
    a. A-B-C-1
    b. A-B+C-1
    c. A+B+C-1
    d. A-B+C

3. How are binary numbers distributed along the number line for the fixed-point number format?
    a. Linearly
    b. Exponentially
    c. Depends on the binary point location
    d. Depends on the exponent

4. For a 6-bit normalized floating-point format: (-1)^S*(.FFF)*(2^EE) where FFFF is unsigned and EE is two's complement, what is the smallest positive value you can represent?
    a. 0.5
    b. 0.25
    c. 0.125
    d. 0.28125

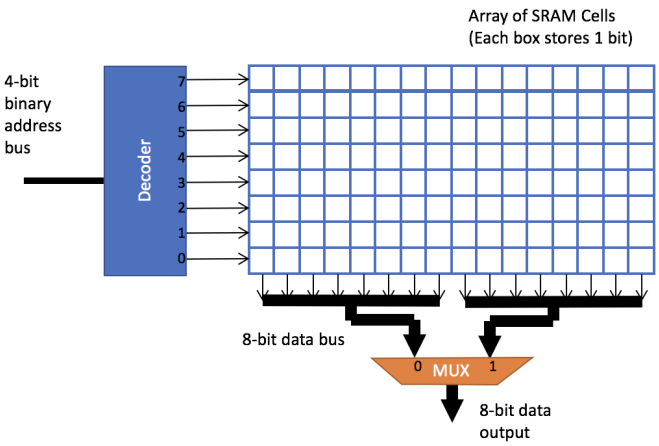
5. Will the results be correct if you use an ALU designed for 32-bit two's complement addition for 32-bit unsigned integers?
    a. Yes, however the overflow flag is calculated differently, and may be incorrect
    b. No, if MSB is 1 for any of the inputs, then the answer will be incorrect
    c. Yes, the same algorithm is used, regardless whether the input happen to be signed or not
    d. No, all signed numbers are subjected to the operation NOT(input) + 1 before an addition, which would produce garbage in the unsigned case

6. A company develops two new types of transistors: LF transistors are very large, but very fast, and SS transistors are very small but slow. How would you design a multiplier for each type of transistor to match its characteristics?
    a. LF: serial, SS: serial
    b. LF: serial, SS: parallel
    c. LF: parallel, SS: parallel
    d. LF: parallel, SS: serial

4. Logic

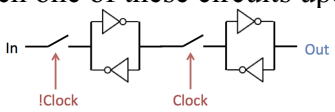
1. Why is DRAM so much cheaper than SRAM?
    a. DRAM doesn't require electricity to retain its values
    b. DRAM memory cells are much smaller than SRAM cells
    c. DRAM doesn't require transistors to work
    d. All of the above

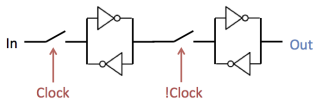
2. What would happen if you replaced the decoder in the SRAM design with an encoder?

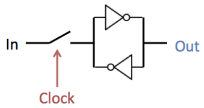


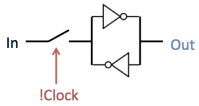
a. It would activate different rows, but it would still work
b. It would activate different rows, and would not work
c. It would activate multiple rows, but would still work
d. It would activate multiple rows, and would not work

3. Which one of these circuits updates its output only when the clock goes from 1 to 0?

a.


b.


c.


d.


a. !Clock / Clock
b. Clock / !Clock
c. Clock
d. !Clock

4. Will a MUX function correctly if more than one input is active (valid) at a time, and why?
a. No, output is undefined or invalid
b. No, a MUX can only have one input at a time
c. Maybe, but it will depend on how the MUX is built internally
d. Yes, output is always the selected input

5. What will be the value of R12 after executing the following two instructions?

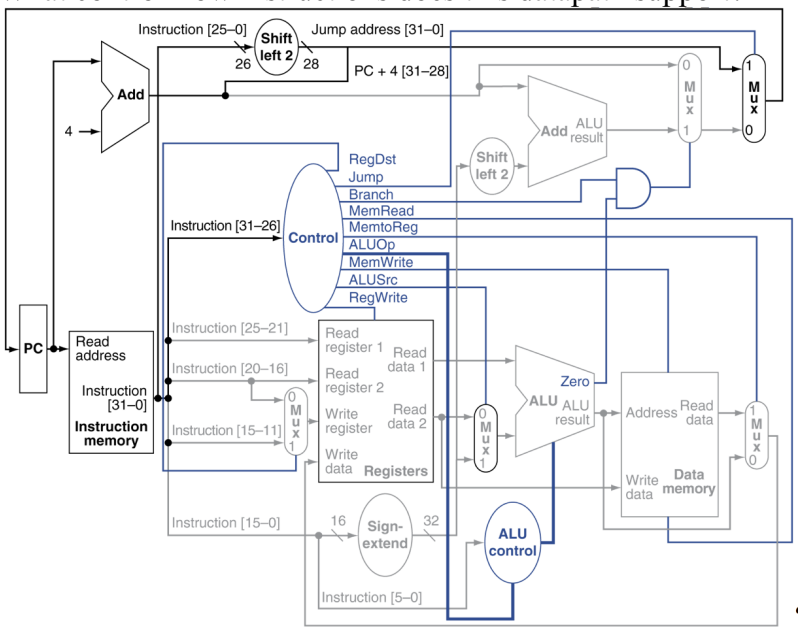
```
and R12, R12, R12
or R12, R12, R12
```

a. 0000 (Zero)
b. FFFF (All ones)
c. It depends on the value of R12 before the first instruction
d. The value of R12 will not change.

6. Why are binary circuits more immune to noise than analog circuit?
    a. Binary values are further apart in terms of voltage than analog values, so they are harder to confuse
    b. Binary circuits generate un-polluted outputs (e.g., 1 or 0 out) after each circuit, so the noise doesn't propagate through
    c. Binary circuits interpret values close to a 1 or 0 as a 1 or 0 even if they are not exactly 1 or 0
    d. All of the above

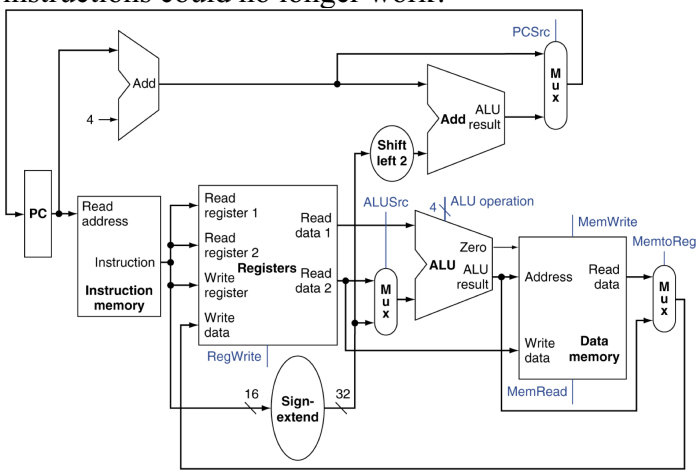
5. Processor Control and Datapath

1. What control flow instructions does this datapath support?



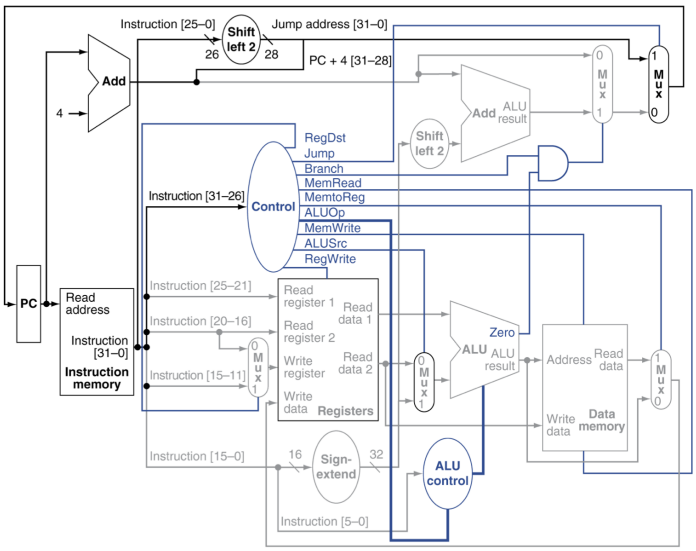
    a. BEQ, BNE
    b. J, BNE
    c. J, BEQ
    d. J, BEQ, BNE

2. If the ALUSrc mux was removed, and Register Read data 2 was connected directly to the ALU, what instructions could no longer work?



    a. R-type
    b. I-type
    c. J-type
    d. All of the above

3. Which part of the processor is storing state?



a. PC, registers, ALU
b. PC, registers, ALU, instruction memory
c. PC, registers, ALU, instruction memory, sign extension
d. PC, registers, instruction memory

4. A single-cycle processor has three types of instructions: A takes 1ns, B takes 2ns, and C takes 10ns. A program with an instruction mix of 25% A, 70% B, and 5% C will run at what clock speed?
   a. 1000 MHz (1 / 1ns)
   b. 465 MHz (1/2.15ns)
   c. 333 MHz (1/3ns)
   d. 100 MHz (1/10ns)

5. What is the logic that determines whether PCSrc should select PC+4 (0) or the ALU result (1)?
   a. (Zero AND bne) OR (Zero AND beq)
   b. (!Zero AND bne) OR (Zero AND beq)
   c. (Zero AND bne) OR (!Zero AND beq)
   d. (!Zero AND bne) OR (!Zero AND beq)

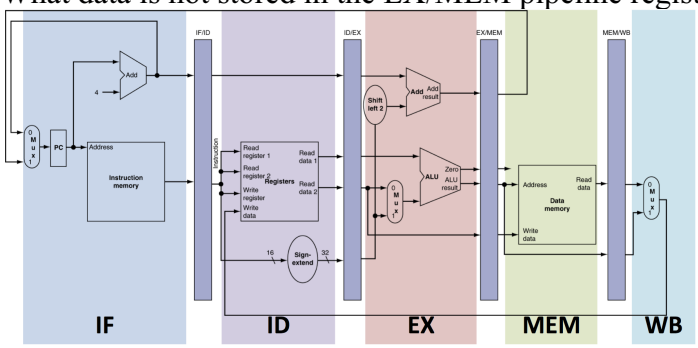
6. A processor has a broken sign-extender that always outputs 0s in the upper 16 bits. How would this limit what it can do?
   a. lw and sw would always use the same address
   b. addi would not change the value
   c. You could not load 32-bit constants
   d. Conditional branches could only jump forwards

6. Pipelining

1. Does the pipelined processor keep the ISA's promise of in-order and atomic execution?
   a. Atomic: yes, In-order: yes
   b. Atomic: yes, In-order: no
   c. Atomic: no, In-order: yes
   d. Atomic: no, In-order: no

2. A single-cycle processor takes 100ns per instruction and can be perfectly divide up into pipeline stages and the pipeline can be kept completely full. However, each pipeline register has an overhead of 1ns. What degree of pipelining gives the best instruction throughput?
   a. 1 stage (1 pipeline register at the end)
   b. 10 stages
   c. 100 stages
   d. 1000 stages

3. What data is not stored in the EX/MEM pipeline register?



   a. Register File 1 output
   b. Register File 2 output
   c. ALU result
   d. Branch address

4. What is the speedup if you have an ideally pipelined processor with 15 stages, but due to control hazards you can only keep it full of instructions 66% of the time?
   a. 0.66x
   b. 10x
   c. 15x
   d. 22.7x

5. Which of the following statements is not true about pipelined processors?
   a. Commercial processors today typically have 15-25 pipeline stages.
   b. It is possible to design processors with multiple parallel pipelines of different lengths.
   c. Doubling the pipeline length doubles the processing speed.
   d. Processors use separate instruction and data caches so they can load an instruction at the same time as they load data.

6. What happens in the MEM stage for an addi instruction?
   a. Nothing
   b. Load the value
   c. Calculate the ALU result
   d. Write back the ALU result

7. Hazards

1. Which of these does not address a structural hazard?
   a. Having independent data and instruction memories
   b. Reading and writing the register file in the same cycle
   c. Extra comparator for branch decisions
   d. Forwarding from a later stage

2. What forwarding paths do we need to execute this code with no NOPs on a processor with a double-pumped register file?

```
add R1, R2, R3
lw R2, 0(R1)
sw R2, 0(R3)
sub R2, R1, R3
```

   a. MEM-EX, WB-EX
   b. MEM-EX, WB-EX, WB-MEM
   c. MEM-EX, WB-MEM
   d. WB-EX, WB-MEM

3. Hazards are a result of what?
    a. Needing to keep the semantics of atomic and sequential execution
    b. Executing instructions in parallel
    c. Executing instructions in pieces
    d. All of the above

4. An 8-stage pipeline processor requires NOPs to avoid hazards. The compiler is aware of this and inserts appropriate NOP instructions into the code. What would happen if the processed code now is run on a single-cycle processor?
    a. The program would execute correctly
    b. Conditional branches would not work correctly, due to the zero output from the ALU
    c. The code would not work correctly because the NOPs are in the wrong place
    d. The code would not work correctly because there are too many NOPs

5. How many register values need to be forwarded in the following code on our standard 5-stage pipeline?
```
add R3, R4, R5
sub R2, R3, R4
beq R2, R3, done
```
    a. 1
    b. 2
    c. 3
    d. 4

6. If a program has 20% branches with 1 branch delay slot per branch, and the compiler can find an instruction to put in the branch delay slot every other time, what is the performance slowdown due to the branch delay slot?
    a. 5%
    b. 10%
    c. 20%
    d. 50%

8. Branch Prediction and Exceptions and Interrupts

1. If you have a branch predictor that keeps track of branch history based on the least significant bits of the PC and can store information for 16 separate branches (direct mapped, 16 entry branch predictor), what happens when you have a program with the following behavior:
branch at 0010 1011 1010 1100 always not taken
branch at 0010 1011 1110 1100 always taken
    a. The program crashes
    b. The branch predictor will only predict for one of the branches
    c. The branch predictor will confuse branches and predict incorrectly
    d. All of the above

2. How many branches are correctly predicted with a 2-bit predictor initialized to strongly not taken?

```
for (i=0; i<100,000; i++) {
        if (i%2==1) { // branch 1: if i is odd
                // do something
        }
} // branch 2: loop
```

    a. branch 1: 50,000/100,000 branch 2: 99,997/100,000
    b. branch 1: 75,000/100,000 branch 2: 50,000/100,000
    c. branch 1: 50,000/100,000 branch 2: 99,998/100,000
    d. branch 1: 99,998/100,000 branch 2: 99,997/100,000

3. Which predictor will not have 50% accuracy for the if statement in this loop?

```
loop {
  i++ // increment variable i
  if (is_odd(i)) // is_odd() returns true if 'i' is odd
    b=b*2 // multiply b by 2;
}
```

   a. 1-bit predictor
   b. 2-bit predictor
   c. BTFNT
   d. Static

4. A program contains 1 million instructions, of which 15% are branches. If we run it on a CPU with a 20 cycles branch penalty and a branch predictor that is 85% accurate, how many cycles will be wasted due to mispredictions?
   a. 100 000 cycles
   b. 200 000 cycles
   c. 450 000 cycles
   d. 1 000 000 cycles

5. What do you have to do to instructions in the pipeline that are executed due to a missprediction?
   a. Nothing if they are memory instructions
   b. Prevent them from changing anything
   c. Insert instructions to reverse their effects
   d. All of the above

6. The branch predictor has a table of multiple branch information. What is the wrong explaination of the information of each branch entry in the predictor?
   a. **PC:** Acts like a 'tag' in a cache. Used to compare with the current PC.
   b. **History:** Holds the information of past branch behavior for this PC
   c. **Valid:** Holds the decision on whether to branch or not for this PC
   d. **Target:** Holds the address to branch to, if the predictor decides to branch

9. Input/Output

1. Why would you use DMA?
   a. Using DMA, the I/O operation can be overlapped with other computation
   b. DMA allows using normal load store instructions to control I/O devices
   c. DMA allows the I/O device to work faster
   d. DMA let's the processor know when the I/O operation is complete

2. Why would you choose to use interrupts instead of polling?
   a. Polling has lower latency than interrupts
   b. Polling uses up less processor time
   c. Interrupts use up less processor time
   d. Interrupts have lower latency

3. Improvement of storage technology has changed I/O. What is <u>incorrect</u> of the following statements?
   a. SSDs have no moving parts resulting in zero 'seek time' unlike spinning drives (HDD)
   b. The lack of seek time in SSDs enables better random I/O (read/write different parts of the disk) compared to HDDs
   c. Because SSDs are generally quicker (in latency) than HDDs, the amount of computation the CPU can do while the disk is working is less for the SSD.
   d. With very fast SSDs (comparable to memory speed), it is beneficial to change from the usual polling to interrupts, as the quicker SSDs will result in shorter interrupt handling times.

4. Which of the following is NOT a benefit of memory-mapped IO?
    a. Faster
    b. Uses regular instructions to access IO
    c. Unifies all the device's memories
    d. Enables the use of virtual memory to protect IO devices

5. What are the benefits of hard disks over flash memory?
    a. Faster
    b. Cheaper
    c. Smaller
    d. None

6. What is hard about building communications busses?
    a. Using enough wires to get the bandwidth
    b. Synchronizing the signals on all the wires
    c. Finding space in servers
    d. All of the above

10. Caches

1. Which property do caches not rely on to work well?
    a. Recently used data will be reused soon
    b. There are more reads than writes
    c. Nearby data will be reused soon
    d. The amount of data used at any one time is small

2. How many lines will each way have in a 256kB, 4-way set associative cache with a 32-byte line size?
    a. 8192
    b. 2048
    c. 1024
    d. 512

3. What is a downside of making the cache line twice as large?
    a. Programs that access data sequentially will waste space
    b. Programs that access data randomly will waste space
    c. Tag comparisons are slower
    d. The percentage of bits spent on tag storage is higher

4. A program is reading every word (4 bytes) consecutively of a very large (4GB) array. Which of the following cache configurations can achieve a hit ratio of more than 50%? Assume LRU replacement policy?
    a. Cache size: 16 B. Block size 4 B. Direct mapped
    b. Cache size: 8 MB. Block size 8 B. 2-way associative
    c. Cache size: 2 KB. Block size 32 B. Fully associative
    d. All of the above

5. Intel's Skylake processors have a 256KB, 16-way set-associative L2 cache with a 64 bytes cache line size. Assuming 32-bits addresses, how are the bits in the address used to access the cache?
    a. 26 bits for the tag, 6 bits to choose a byte within a cache line
    b. 6 bits for the tag, 18 bits for the index, 8 bits to choose a byte within a cache line
    c. 8 bits for the tag, 18 bits for the index, 6 bits to choose a byte within a cache line
    d. 18 bits for the tag, 8 bits for the index, 6 bits to choose a byte within a cache line

6. What is the incorrect description about cache organization?
    a. Direct mapped caches result in more conflict misses, meaning that data at two different addresses may map to the same entry and push each other out.
    b. N-way Set associative caches can help solve the conflict miss of direct-mapped caches by putting N blocks in the same set.
    c. Fully-associative caches are the cheapest to access, as they only need one tag comparison per cache lookup
    d. Direct mapped caches do not need replacement policies, but set associative and fully associative caches do

11. Virtual Memory

1. Which TLB/cache arrangement allows you to know if you have a hit in the cache without checking the TLB?
    a. Virtually indexed, virtually tagged
    b. Virtually indexed, physically tagged
    c. Physically indexed, virtually tagged
    d. Physically indexed, physically tagged

2. Which of the following statements is **not** true about virtual memory?
    a. VM provides security by allowing other programs to read each other's data.
    b. VM allows programs to use more memory than is physically available in the system.
    c. VM saves memory space by allowing programs to share common data instead of each program having their private copies.
    d. VM provides flexibility by using indirection to use non-contiguous physical memory in a contiguous virtual memory space.

3. Why would we prefer a single-level page table rather than a multi-level one?
    a. To reduce the total size of the page table
    b. To make translations faster
    c. To reduce the in-memory size of the page table
    d. None of the above

4. What is/are the biggest reasons that are page faults so excruciatingly slow?
    a. The TLB has to be reloaded with the right page table data
    b. The data has to be accessed from the hard disk
    c. The OS has to choose which data to evict from DRAM
    d. All of the above

5. What is *not* a benefit of virtual memory?
    a. Memory is protected from other applications
    b. Memory accesses are faster
    c. Memory appears larger than the installed DRAM
    d. Memory does not have to be used in contiguous chunks

6. What does the TLB do?
    a. It caches recently-used data
    b. It caches recently-used address translations
    c. It caches recently-used pages
    d. It caches recently-used instructions

12. Parallelism

1. What does a lock do?
   a. Protect data so only one processor can access it at a time
   b. Synchronize data so that different caches see the same value
   c. Provide a way for a program to check if another processor is using the data
   d. All of the above

2. The fastest computer in Sweden, Beskow at KTH, has 53,632 processor cores. What would be the speedup of a program that was 99.99% parallel?
   a. 5x
   b. 536x
   c. 8,400x
   d. 53,632x

3. My laptop has 2 CPU processor cores that run at 2GHz (billion cycles per second) and can each do 4 instructions per cycle and a GPU (graphics processor) with 16 processor cores that run at 1GHz and each do 2 instructions per cycle. You can use either the CPU or the GPU at any given time. What is the fastest rate I can run a program that is 50% parallelizable in GIPS (giga (billion) instructions per second)?
   a. 12 GIPS
   b. 17 GIPS
   c. 20 GIPS
   d. 32 GIPS

4. The cache for processor 2 wants to load an address. What does the cache for processor 1 need to check to see if it needs to write back data and tell processor 2's cache to wait for it?
   a. If it has the line (tag and valid) and it is dirty
   b. If it has the line (tag and valid) and it is not dirty
   c. If it has the line (tag and valid)
   d. If it has the dirty line (tag and dirty)

5. Which of the following designs to improve performance does not require multiple program counters (PC)?
   a. Simultaneous multithreading (issuing instructions from multiple programs into the pipeline at the same time)
   b. Multicore processors (having multiple processors on one chip)
   c. Dual-issue processors (issuing multiple instructions from the same program into the pipeline at the same time)
   d. All of them require multiple PCs

6. A processor can do two ALU operations (add/sub/etc.) at the same time for two instructions that come one after the other. A program has 40% load/stores, and 60% ALU operations. 10% of the instructions depend on a subsequent instruction. What is the best IPC you would expect?
   a. 1.5
   b. 1.6
   c. 1.8
   d. 1.9

Make sure you circle the answers on the answer page.
Answers on other pages will NOT be graded.